	`Pick`, `Record`, `Exclude`.	domain models.	advanced scenarios like deep partials, recursive types, and API inference.

Now that functions, classes, and advanced types in TypeScript are clear, we can dive into interview questions to see how these ideas are applied.

Question: "What are generics in TypeScript?"

Analysis: This question tests a candidate's understanding of how TypeScript uses generics to write flexible, reusable, and type-safe code. It evaluates whether the candidate can explain the purpose of generics, how they differ from using `any`, and how they are applied in real-world scenarios such as components, functions, and classes.

Answer: Generics in TypeScript allow the creation of components, functions, or classes that can work with a variety of types rather than just one. Instead of specifying a concrete type, a type variable (commonly named T, U, K, or V) is used as a placeholder. This placeholder is replaced with an actual type when the code is used, making code more reusable and flexible without sacrificing type safety.

Unlike using `any`, which bypasses type checking and reduces safety, generics maintain strict type information throughout the code. Listing 3.15 demonstrates generics in React with TypeScript. In this example, the `ListProps<T>` interface is crucial because it defines a contract for the props that the generic `List` component expects. It specifies that the `List` component will receive an array of items of type T and a function that knows how to render each item of that type into a JSX element. This setup allows the component to be flexible and reusable while maintaining full type safety.

Listing 3.15 Generic List component

```
interface ListProps<T> { #A
  items: T[]; #B
  renderItem: (item: T) => JSX.Element; #C
}
const List = <T,>({ items, renderItem }: ListProps<T>) => (
```